

```
self, datadir, ndims):
    s.path.join(datadir, "id.txt")
    = [x.strip() for x in str.split(open(idfil
index = dict(zip(self.names, range(len(sel
s = ndims
urefile = os.path.join(datadir, "feature.b
BigFile] %d features, %d dimensions" % (len
binary: %s" % self.featurefile
txt: %s" % idfile

self, requested, isname=True):
    me:
    dex_name_array = [(self.name2index[x], x)
    assert(min(requested)>=0)
    assert(max(requested)<len(self.names))
    dex_name_array = [(x, self.names[x]) for
    dex_name_array.sort()

    = seq_read(self.featurefile, self.ndims,
    return [x[1] for x in dex_name_array], vec

    ps(self):
    return [len(self.names), self.ndims]
```

Iterators, Iterables and Generators in Python

Discover the power of iterators and generators in Python and learn how to leverage the itertools package for enhanced functionality in your code.



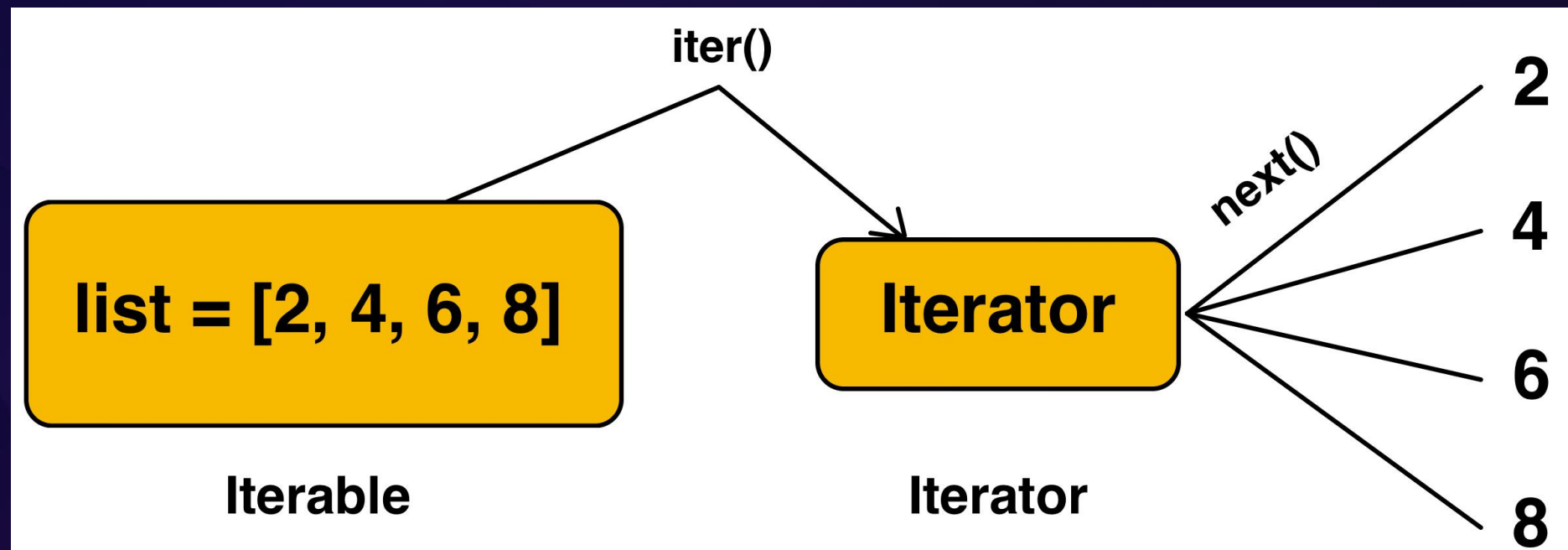
by Mohd Iqbal Bhat

What are Iterables?

- An iterable is an object that can be iterated over, meaning that you can access its elements one at a time.
- An iterable is simply a Python object which consists of a collection of data members. We can access the data members of an iterable one at a time. An iterable can be “iterated over” with a loop, like a for loop in Python.
- Iterables do not have any state on their own; they simply provide a way to create an iterator.
- Examples:
 - Lists
 - Strings
 - Tuples
 - Sets
 - Dictionaries
- Functions that work with Iterables:
 - `sum()`
 - `sorted()`
 - `max()`
 - `min()`

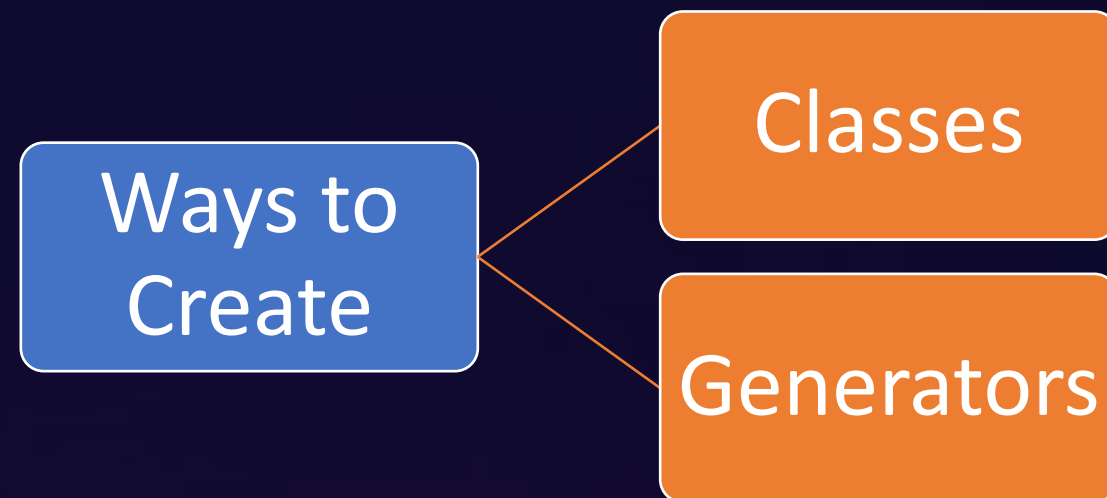
What are Iterators?

- An iterator is an object that represents a stream of data.
- It keeps track of its position in the stream and allows you to access the next element in the stream.
- Iterators are created from iterables using the `iter()` function.
- The `iter()` method is used to initialize the iterator object.
- Iteration is accomplished through the usage of the `next()` method.
- Example:
 - `list = [2,4,6,8]`
 - `My_iterator = iter(list)`



Iterators implementation:

- Python iterator implements the iterator protocol, which includes two special methods: `__iter__()` and `__next__()`.
- The `__iter__()` method – returns an iterator object. This is required in order to use containers and iterators with the `for` and `in` statements.
- The `__next__()` method – This function returns the sequence's next item. If no more items are found, throw the **StopIteration** exception.



Iterator Using Classes

1

Creating an Iterator

Define your own class with functions `__iter__()` and `__next__()` functions.

2

Exception Handling

Handle the `StopIteration` exception and gracefully exit your iterator.

Iterators implementation using Classes:

```
class Squares:
    def __init__(self, length):
        self.length = length
        self.current = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.current >= self.length:
            raise StopIteration
        self.current += 1
        return self.current ** 2

square_num = Squares(5)
for sq in square_num:
    print(sq)
print(next(square_num))
```

Generators

1

Creating a Generator Function

Master the art of creating generator functions using the `yield` keyword and understand their behavior.

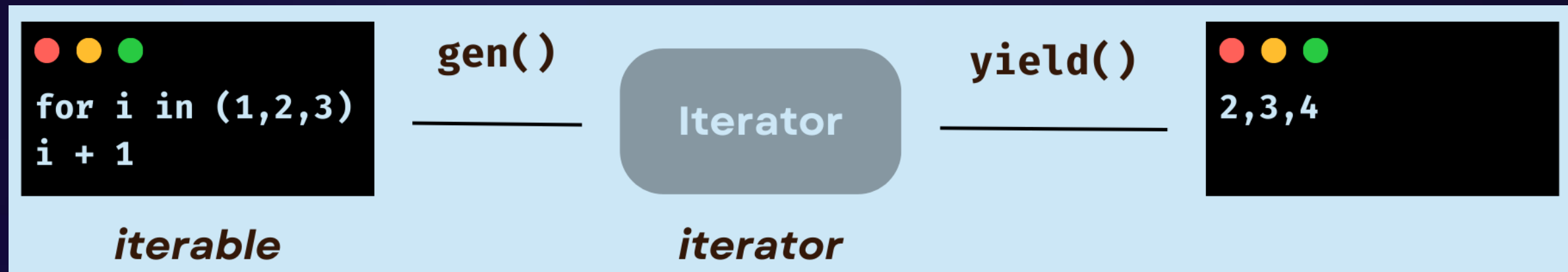
2

Memory Efficiency

Explore how generators offer significant memory efficiency benefits, especially when dealing with large datasets.

What are Generators?

- A generator is a special type of function that produces a sequence of values without creating a complete list in memory.
- Generators are useful for generating large sequences of data, such as infinite sequences or sequences that are too large to fit into memory at once.



Creating Generators?

- Generators are created using the `def` keyword, just like regular functions. However, instead of using the `return` statement, generators use the `yield` statement to produce a value from the sequence.
- The `yield` statement pauses the execution of the generator function and returns the value to the caller.
- When the caller requests the next value from the generator, the generator function resumes execution from the point where it left off.

```
def squares():  
    for number in range(1, 11):  
        yield number * number
```

Executed at 2023.11.14 09:26:54 in 137ms

```
for square in squares():  
    print(square)
```

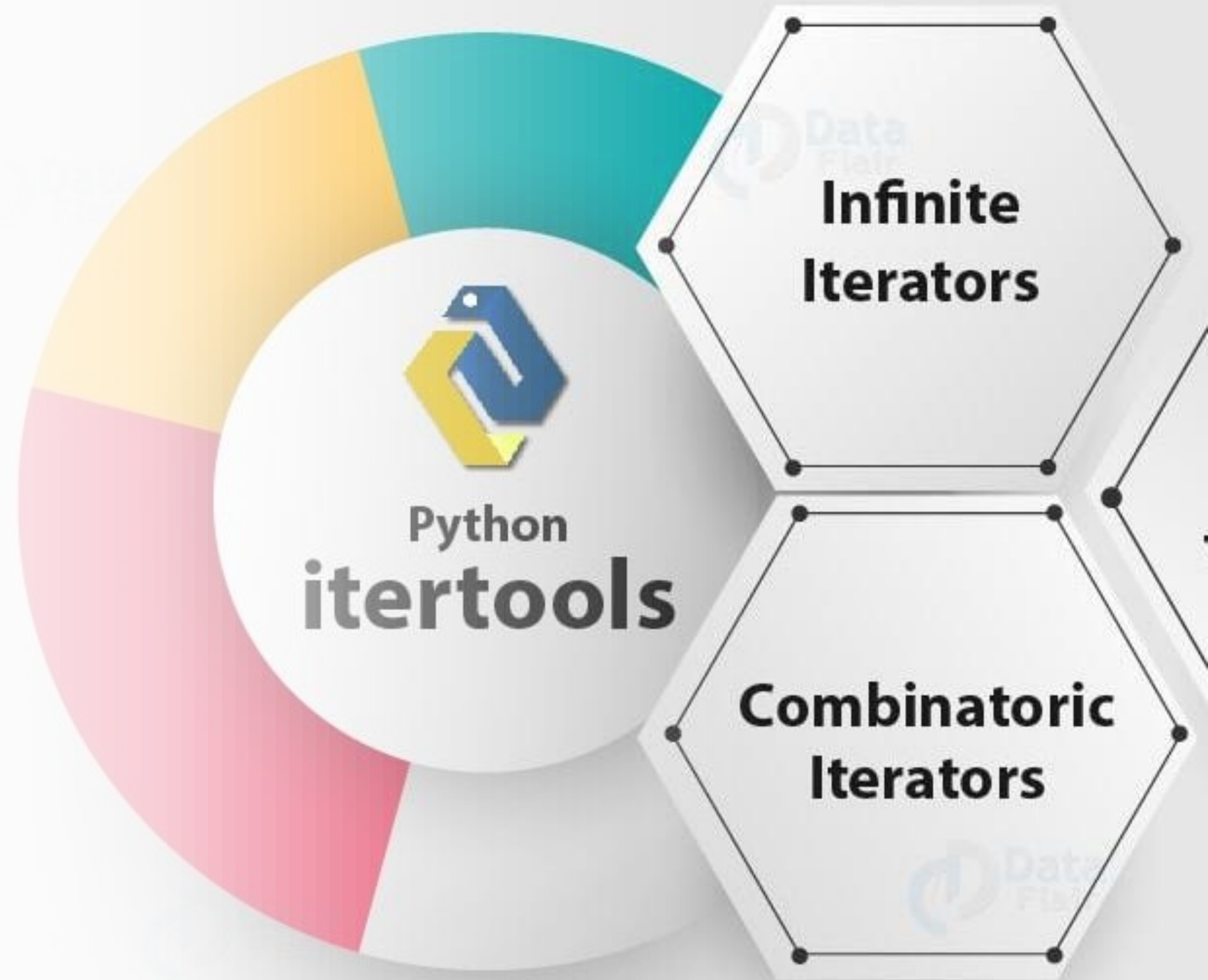
Benefits of Generators?

- **Memory efficiency:** Generators do not create a complete list of values in memory. Instead, they produce values one at a time, which can be a significant savings for large sequences.
- **Laziness:** Generators are lazy, meaning that they do not start generating values until they are actually requested. This can be useful for sequences that are only partially needed or that depend on other computations.
- **Flexibility:** Generators can be used to produce a wide variety of sequences, including infinite sequences and sequences that are generated from external sources.

Applications of Generators?

- **File processing:** Generators can be used to read files one line at a time, which is more efficient than reading the entire file into memory at once.
- **Web scraping:** Generators can be used to scrape data from websites one page at a time, which is more efficient than scraping the entire website at once.
- **Data streaming:** Generators can be used to stream data from a source, such as a sensor or a database, to a destination, such as a file or a database.
- **Algorithms:** Generators can be used to implement algorithms that generate sequences of values, such as Fibonacci sequences or prime number generators.

Itertools package



Itertools package?

- The Python itertools module has functions for creating iterators for efficient looping.
- It provides a set of fast, memory-efficient tools for handling iterators.
- It is a part of the standard library and offers a collection of functions that can be used to create and work with iterators.
- While some iterators are infinite, some terminate on the shortest input sequence. Yet, some are combinatoric.

A. Infinite Iterators

count(start=0, step=1)

The `count()` function returns a sequence of values starting from a specified value (start) and incrementing by a specified step size (step).

```
>>> from itertools import count
>>> for i in count(10,2):
    print(i)
    if i>25: break
```

cycle(iterable)

The `cycle()` function creates an iterator that cycles through the elements of an iterable indefinitely.

```
>>> from itertools import cycle
>>> for i in cycle(['red','green','blue']):
    print(i)
```

repeat(elem [,n])

The `repeat()` function repeats a specified element (elem) either a specified number of times (n) or endlessly into the iterator.

```
>>> from itertools import repeat
>>> for i in repeat('Red',3):
```


B. Combinatoric Iterators

`product(*iterables, repeat=1)`

The `product()` function returns the cartesian product of the input iterables. It generates all possible combinations of elements, similar to nested for-loops.

```
>>> for i in product([1,2,3],[4,5,6]):  
    print(i)
```

`permutations(iterable, r=None)`

The `permutations()` function generates all possible permutations of a specified length (`r`) from the elements of an iterable. The permutations are generated in lexicographic order, without any repetition of elements.

```
>>> from itertools import permutations  
>>> for i in permutations('ABCD'):  
    print(i)
```

`combinations(iterable, r)`

The `combinations()` function returns all possible subsequences of length `r` from the elements of an iterable.

```
>>> from itertools import combinations  
>>> for i in combinations('ABCD',2):  
    print(i)
```

`combinations_with_replacement(iterable, r)`

This returns `r`-length subsequences of elements of the iterable, where individual elements may repeat.

```
>>> from itertools import  
combinations_with_replacement as cwr  
>>> for i in cwr('ABCD',2):  
    print(i)
```

C. Terminating Iterators

1

`accumulate(iterable [,func])`

The `accumulate()` function creates an iterator that yields accumulated sums or results of a specified binary function.

```
import operator
```

```
>>> for i in accumulate([1,2,3,4,5],operator.mul):  
    print(i)
```

2

`chain(*iterables)`

The `chain()` function creates an iterator that combines elements from multiple iterables. It moves to the next iterable once the current one is exhausted.

```
>>> from itertools import chain
```

```
>>> for i in chain('Hello','World','Bye'):  
    print(i)
```

3

`chain.from_iterable(iterable)`

The `chain.from_iterable()` function is an alternative constructor to `chain()`. It takes a single iterable argument and lazily evaluates the chained inputs.

```
>>> for i in chain.from_iterable(['Hello','World','Bye']):
```

4

`compress(data, selectors)`

The `compress()` function filters elements from the data iterable based on the corresponding values in the selectors iterable.

```
>>> from itertools import compress
```

```
>>> for i in compress('ABCDEF',[1,0,1,True,0,'']):  
    print(i)
```

5

`dropwhile(predicate, iterable)`

The `dropwhile()` function drops elements from the iterable as long as the specified predicate function returns True. Once the predicate returns False, it starts returning every element.

```
>>> from itertools import dropwhile
```

```
>>> for i in dropwhile(lambda x:x<7,[1,2,7,9,5,3,2,9]):  
    print(i)
```

Conclusion

The Python Itertools package provides a wide range of iterator-generating functions categorized into infinite iterators, combinatoric iterators, and terminating iterators. By understanding and utilizing these iterators, you can enhance your coding efficiency and tackle a variety of programming challenges.

```
print "Hello World"
```